

CS 224n Spring 2024: Assignment #3

Due Date: April 30th, Tuesday, 4:30 PM PST.

This assignment is split into two sections: *Neural Machine Translation with RNNs* and *Analyzing NMT Systems*. The first is primarily coding and implementation focused, whereas the second entirely consists of written, analysis questions. If you get stuck on the first section, you can always work on the second as the two sections are independent of each other. Note that the NMT system is more complicated than the neural networks we have previously constructed within this class and takes about **2 hours to train on a GPU**. Thus, we strongly recommend you get started early with this assignment. Finally, the notation and implementation of the NMT system is a bit tricky, so if you ever get stuck along the way, please come to Office Hours so that the TAs can support you.

1. Neural Machine Translation with RNNs (45 points)

In Machine Translation, our goal is to convert a sentence from the *source* language (e.g. Mandarin Chinese) to the *target* language (e.g. English). In this assignment, we will implement a sequence-to-sequence (Seq2Seq) network with attention, to build a Neural Machine Translation (NMT) system. In this section, we describe the **training procedure** for the proposed NMT system, which uses a Bidirectional LSTM Encoder and a Unidirectional LSTM Decoder.

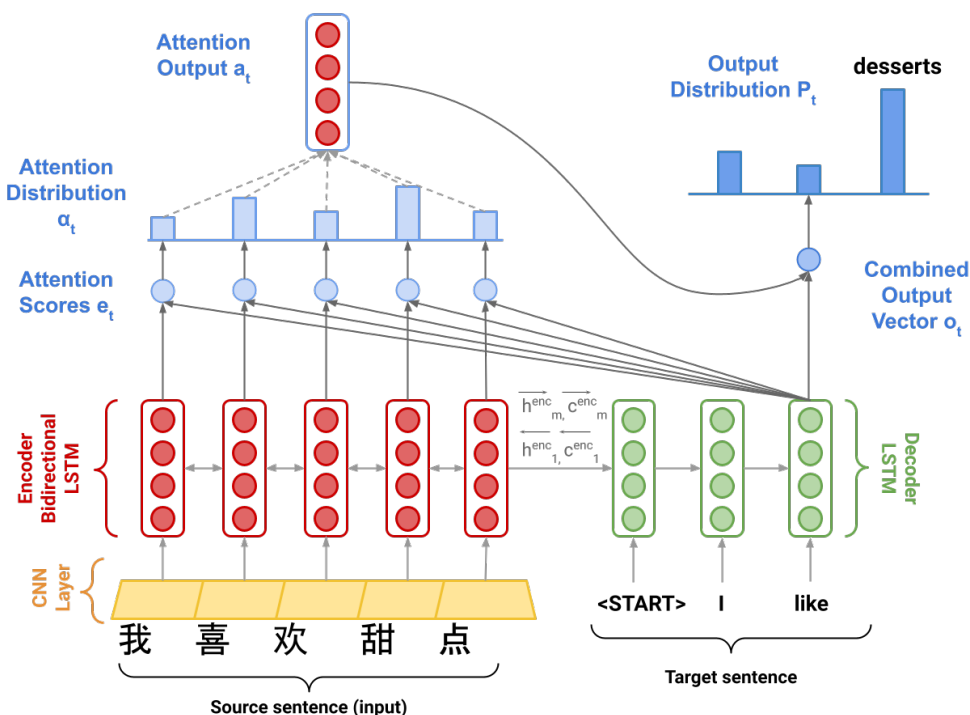


Figure 1: Seq2Seq Model with Multiplicative Attention, shown on the third step of the decoder. Hidden states h_i^{enc} and cell states c_i^{enc} are defined on the next page.

Model description (training procedure)

Given a sentence in the source language, we look up the character or word embeddings from an **embeddings matrix**, yielding $\mathbf{x}_1, \dots, \mathbf{x}_m$ ($\mathbf{x}_i \in \mathbb{R}^{e \times 1}$), where m is the length of the source sentence and e is

the embedding size. We then feed the embeddings to a **convolutional layer**¹ while maintaining their shapes. We feed the convolutional layer outputs to the **bidirectional encoder**, yielding hidden states and cell states for both the forwards ($\rightarrow$) and backwards ($\leftarrow$) LSTMs. The forwards and backwards versions are concatenated to give hidden states $\mathbf{h}_i^{\text{enc}}$ and cell states $\mathbf{c}_i^{\text{enc}}$:

$$\mathbf{h}_i^{\text{enc}} = [\overleftarrow{\mathbf{h}_i^{\text{enc}}}; \overrightarrow{\mathbf{h}_i^{\text{enc}}}] \text{ where } \mathbf{h}_i^{\text{enc}} \in \mathbb{R}^{2h \times 1}, \overleftarrow{\mathbf{h}_i^{\text{enc}}}, \overrightarrow{\mathbf{h}_i^{\text{enc}}} \in \mathbb{R}^{h \times 1} \quad 1 \leq i \leq m \quad (1)$$

$$\mathbf{c}_i^{\text{enc}} = [\overleftarrow{\mathbf{c}_i^{\text{enc}}}; \overrightarrow{\mathbf{c}_i^{\text{enc}}}] \text{ where } \mathbf{c}_i^{\text{enc}} \in \mathbb{R}^{2h \times 1}, \overleftarrow{\mathbf{c}_i^{\text{enc}}}, \overrightarrow{\mathbf{c}_i^{\text{enc}}} \in \mathbb{R}^{h \times 1} \quad 1 \leq i \leq m \quad (2)$$

We then initialize the **decoder's** first hidden state $\mathbf{h}_0^{\text{dec}}$ and cell state $\mathbf{c}_0^{\text{dec}}$ with a linear projection of the encoder's final hidden state and final cell state.²

$$\mathbf{h}_0^{\text{dec}} = \mathbf{W}_h [\overleftarrow{\mathbf{h}_1^{\text{enc}}}; \overrightarrow{\mathbf{h}_m^{\text{enc}}}] \text{ where } \mathbf{h}_0^{\text{dec}} \in \mathbb{R}^{h \times 1}, \mathbf{W}_h \in \mathbb{R}^{h \times 2h} \quad (3)$$

$$\mathbf{c}_0^{\text{dec}} = \mathbf{W}_c [\overleftarrow{\mathbf{c}_1^{\text{enc}}}; \overrightarrow{\mathbf{c}_m^{\text{enc}}}] \text{ where } \mathbf{c}_0^{\text{dec}} \in \mathbb{R}^{h \times 1}, \mathbf{W}_c \in \mathbb{R}^{h \times 2h} \quad (4)$$

With the decoder initialized, we must now feed it a target sentence. On the t^{th} step, we look up the embedding for the t^{th} subword, $\mathbf{y}_t \in \mathbb{R}^{e \times 1}$. We then concatenate $\mathbf{y}_t$ with the *combined-output vector* $\mathbf{o}_{t-1} \in \mathbb{R}^{h \times 1}$ from the previous timestep (we will explain what this is later down this page!) to produce $\overline{\mathbf{y}}_t \in \mathbb{R}^{(e+h) \times 1}$. Note that for the first target subword (i.e. the start token) $\mathbf{o}_0$ is a zero-vector. We then feed $\overline{\mathbf{y}}_t$ as input to the decoder.

$$\mathbf{h}_t^{\text{dec}}, \mathbf{c}_t^{\text{dec}} = \text{Decoder}(\overline{\mathbf{y}}_t, \mathbf{h}_{t-1}^{\text{dec}}, \mathbf{c}_{t-1}^{\text{dec}}) \text{ where } \mathbf{h}_t^{\text{dec}} \in \mathbb{R}^{h \times 1}, \mathbf{c}_t^{\text{dec}} \in \mathbb{R}^{h \times 1} \quad (5)$$

$$(6)$$

We then use $\mathbf{h}_t^{\text{dec}}$ to compute multiplicative attention over $\mathbf{h}_1^{\text{enc}}, \dots, \mathbf{h}_m^{\text{enc}}$:

$$\mathbf{e}_{t,i} = (\mathbf{h}_t^{\text{dec}})^T \mathbf{W}_{\text{attProj}} \mathbf{h}_i^{\text{enc}} \text{ where } \mathbf{e}_t \in \mathbb{R}^{m \times 1}, \mathbf{W}_{\text{attProj}} \in \mathbb{R}^{h \times 2h} \quad 1 \leq i \leq m \quad (7)$$

$$\alpha_t = \text{softmax}(\mathbf{e}_t) \text{ where } \alpha_t \in \mathbb{R}^{m \times 1} \quad (8)$$

$$\mathbf{a}_t = \sum_{i=1}^m \alpha_{t,i} \mathbf{h}_i^{\text{enc}} \text{ where } \mathbf{a}_t \in \mathbb{R}^{2h \times 1} \quad (9)$$

$\mathbf{e}_{t,i}$ is a scalar, the i th element of $\mathbf{e}_t \in \mathbb{R}^{m \times 1}$, computed using the hidden state of the decoder at the t th step, $\mathbf{h}_t^{\text{dec}} \in \mathbb{R}^{h \times 1}$, the attention projection $\mathbf{W}_{\text{attProj}} \in \mathbb{R}^{h \times 2h}$, and the hidden state of the encoder at the i th step, $\mathbf{h}_i^{\text{enc}} \in \mathbb{R}^{2h \times 1}$.

We now concatenate the attention output $\mathbf{a}_t$ with the decoder hidden state $\mathbf{h}_t^{\text{dec}}$ and pass this through a linear layer, tanh, and dropout to attain the *combined-output vector* $\mathbf{o}_t$.

$$\mathbf{u}_t = [\mathbf{a}_t; \mathbf{h}_t^{\text{dec}}] \text{ where } \mathbf{u}_t \in \mathbb{R}^{3h \times 1} \quad (10)$$

$$\mathbf{v}_t = \mathbf{W}_u \mathbf{u}_t \text{ where } \mathbf{v}_t \in \mathbb{R}^{h \times 1}, \mathbf{W}_u \in \mathbb{R}^{h \times 3h} \quad (11)$$

$$\mathbf{o}_t = \text{dropout}(\tanh(\mathbf{v}_t)) \text{ where } \mathbf{o}_t \in \mathbb{R}^{h \times 1} \quad (12)$$

¹Checkout <https://cs231n.github.io/convolutional-networks> for an in-depth description for convolutional layers if you are not familiar

²If it's not obvious, think about why we regard $[\overleftarrow{\mathbf{h}_1^{\text{enc}}}; \overrightarrow{\mathbf{h}_m^{\text{enc}}}]$ as the 'final hidden state' of the Encoder.

Then, we produce a probability distribution $\mathbf{P}_t$ over target subwords at the t^{th} timestep:

$$\mathbf{P}_t = \text{softmax}(\mathbf{W}_{\text{vocab}} \mathbf{o}_t) \text{ where } \mathbf{P}_t \in \mathbb{R}^{V_t \times 1}, \mathbf{W}_{\text{vocab}} \in \mathbb{R}^{V_t \times h} \quad (13)$$

Here, V_t is the size of the target vocabulary. Finally, to train the network we then compute the cross entropy loss between $\mathbf{P}_t$ and $\mathbf{g}_t$, where $\mathbf{g}_t$ is the one-hot vector of the target subword at timestep t :

$$J_t(\theta) = \text{CrossEntropy}(\mathbf{P}_t, \mathbf{g}_t) \quad (14)$$

Here, θ represents all the parameters of the model and $J_t(\theta)$ is the loss on step t of the decoder. Now that we have described the model, let's try implementing it for Mandarin Chinese to English translation!

Setting up Your Local Development Environment

Ensure that you have [conda](#) installed. Unzip the starter code, and open the resulting directory in your favorite python integrated development environment ([Visual Studio Code](#) is a popular choice). Open a terminal and navigate (`cd`) to the directory that contains the `env-cpu.yml` and `env-gpu.yml` files. Create and activate the `cs224n-cpu` conda environment as follows:

```
conda env create --file env-cpu.yml
conda activate cs224n-cpu
```

Setting up Your Cloud GPU-powered Virtual Machine

Follow the instructions in the [GCP Guide for CS224n](#) (link also provided on website and Ed) to create your VM instance. This should take you approximately 45 minutes. Though you will need the GPU to train your model, we strongly advise that you first develop the code locally and ensure that it runs, before attempting to train it on your VM. GPU time is expensive and limited. It takes approximately **1.5 to 2 hours** to train the NMT system. We don't want you to accidentally use all your GPU time for debugging your model rather than training and evaluating it. Finally, **make sure that your VM is turned off whenever you are not using it.**

If your GCP subscription runs out of money, your VM will be temporarily locked and inaccessible. If that happens, please make a private Ed post describing the situation.

After setting up the cloud VM, you should turn it off while you work on the programming assignment locally.

Implementation and written questions

- (a) (2 points) (coding) In order to apply tensor operations, we must ensure that the sentences in a given batch are of the same length. Thus, we must identify the longest sentence in a batch and pad others to be the same length. Implement the `pad_sents` function in `utils.py`, which shall produce these padded sentences.
- (b) (3 points) (coding) Implement the `__init__` function in `model_embeddings.py` to initialize the necessary source and target embeddings.
- (c) (4 points) (coding) Implement the `__init__` function in `nmt_model.py` to initialize the necessary model layers (LSTM, CNN, projection, and dropout) for the NMT system.

- (d) (8 points) (coding) Implement the `encode` function in `nmt_model.py`. This function converts the padded source sentences into the tensor $\mathbf{X}$, generates $\mathbf{h}_1^{\text{enc}}, \dots, \mathbf{h}_m^{\text{enc}}$, and computes the initial state $\mathbf{h}_0^{\text{dec}}$ and initial cell $\mathbf{c}_0^{\text{dec}}$ for the Decoder. You can run a non-comprehensive sanity check by executing:

```
python sanity_check.py 1d
```

- (e) (8 points) (coding) Implement the `decode` function in `nmt_model.py`. This function constructs $\bar{\mathbf{y}}$ and runs the `step` function over every timestep for the input. You can run a non-comprehensive sanity check by executing:

```
python sanity_check.py 1e
```

- (f) (10 points) (coding) Implement the `step` function in `nmt_model.py`. This function applies the Decoder's LSTM cell for a single timestep, computing the encoding of the target subword $\mathbf{h}_t^{\text{dec}}$, the attention scores $\mathbf{e}_t$, attention distribution α_t , the attention output $\mathbf{a}_t$, and finally the combined output $\mathbf{o}_t$. You can run a non-comprehensive sanity check by executing:

```
python sanity_check.py 1f
```

- (g) (3 points) (written) The `generate_sent_masks()` function in `nmt_model.py` produces a tensor called `enc_masks`. It has shape (batch size, max source sentence length) and contains 1s in positions corresponding to 'pad' tokens in the input, and 0s for non-pad tokens. Look at how the masks are used during the attention computation in the `step()` function (lines 295-296).

First explain (in around three sentences) what effect the masks have on the entire attention computation. Then explain (in one or two sentences) why it is necessary to use the masks in this way.

Solution: *Effect on the entire attention computation.* The masks indicate which source positions are padding tokens: `enc_masks` holds a 1 exactly at every `<pad>` position and a 0 at every real token. Inside `step`, the line `e_t.data.masked_fill(enc_masks.bool(), -float('inf'))` overwrites the attention score $e_{t,i}$ at every padding position with $-\infty$ before the softmax is applied. Because $\text{softmax}(-\infty)_i = 0$, every padding position receives exactly zero attention weight, so the resulting distribution α_t places all of its mass on the genuine source tokens and the probability that would otherwise have leaked onto pads is redistributed over the real words. As a result, the attention output $\mathbf{a}_t = \sum_i \alpha_{t,i} \mathbf{h}_i^{\text{enc}}$ becomes a weighted combination of only the meaningful encoder hidden states.

Why it is necessary. Without the masks, the decoder could attend to `<pad>` tokens, which carry no linguistic content (their hidden states produced by the packed LSTM are effectively meaningless), thereby wasting attention mass and injecting noise into the context vector. Masking forces the model to restrict its attention to the actual source words, keeping the attention weights a valid, normalized distribution over real content and improving translation quality.

Now it's time to get things running! As noted earlier, we recommend that you develop the code on your personal computer. Confirm that you are running in the proper conda environment and then execute the following command to train the model on your local machine:

```
sh run.sh train_local
(Windows) run.bat train_local
```

To help with monitoring and debugging, the starter code uses tensorboard to log loss and perplexity during training using TensorBoard³. TensorBoard provides tools for logging and visualizing training information from experiments. To open TensorBoard and monitor the training process, run the following

³<https://pytorch.org/docs/stable/tensorboard.html>

in a separate terminal in which you have also activated the `cs224n-cpu` conda environment, then access tensorboard at <http://localhost:6006/>.

```
tensorboard --logdir=runs --port 6006
```

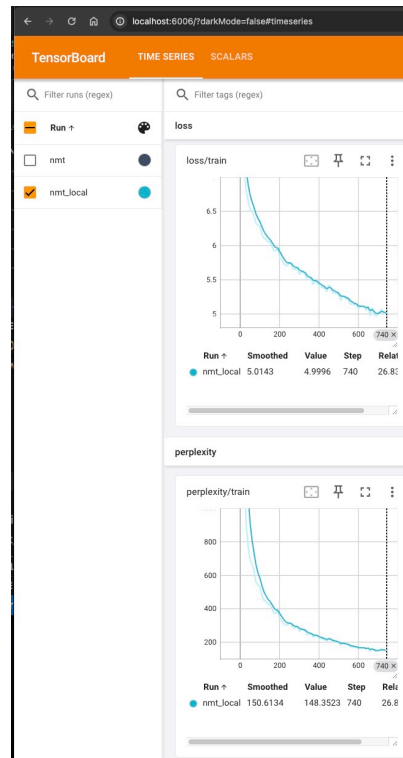


Figure 2: Tensorboard showing loss and perplexity values on your local machine

You should see a significant decrease in loss during the initial iterations (Fig 2). Once your code runs for a few hundreds of iterations without crashing, power on your VM from the GCP Console.

Train the NMT system on the VM: Refer to the GCP How-to Appendix to:

- Setup your GCP VM and Obtain SSH Connection Info
 - Connect to the VM (with SSH Tunneling setup so that you can view remote tensorboard logs)
 - Copy your code from your computer to the cloud VM
 - Train the NMT System on the Cloud VM
 - Download the Gradescope Submission Package from Your Cloud VM
- (h) (3 points) (written) Once your model is done training (**this should take under 2 hours on the VM**), execute the following command to test the model:

```
sh run.sh test
```

Please report the model's corpus BLEU Score. It should be larger than 18.

Solution: As a self-learner without GPU resources, I have not yet trained the full NMT system on the training set, so I cannot report a corpus BLEU score at this time. Once the model is trained on a GPU (expected to take under 2 hours) and evaluated with `sh run.sh test`, the corpus BLEU score should exceed 18. Parts 1a–1f have been validated via the provided sanity checks and a CPU smoke test (training loss decreasing), so the implementation is ready to be trained whenever GPU access becomes available.

- (i) (4 points) (written) In class, we learned about dot product attention, multiplicative attention, and additive attention. As a reminder, dot product attention is $\mathbf{e}_{t,i} = \mathbf{s}_t^T \mathbf{h}_i$, multiplicative attention is $\mathbf{e}_{t,i} = \mathbf{s}_t^T \mathbf{W} \mathbf{h}_i$, and additive attention is $\mathbf{e}_{t,i} = \mathbf{v}^T \tanh(\mathbf{W}_1 \mathbf{h}_i + \mathbf{W}_2 \mathbf{s}_t)$.

- i. (2 points) Explain one advantage and one disadvantage of *dot product attention* compared to multiplicative attention.

Solution: Dot product vs. multiplicative.

Advantage: dot product attention has no extra learned parameter matrix $\mathbf{W}$, so it is simpler, cheaper to compute, and requires fewer parameters and less memory; it also reduces to a single inner product rather than a matrix multiplication.

Disadvantage: because it uses the identity matrix in place of $\mathbf{W}$, it cannot learn to re-weight or rotate the query/key interaction, so multiplicative attention with a learned $\mathbf{W}$ can capture a richer, more flexible relationship between $\mathbf{s}_t$ and $\mathbf{h}_i$; dot product is therefore less expressive, and its scores can also grow with the dimension unless scaled.

- ii. (2 points) Explain one advantage and one disadvantage of *additive attention* compared to multiplicative attention.

Solution: Additive vs. multiplicative.

Advantage: additive attention passes the projected query and key through learned projections $\mathbf{W}_1, \mathbf{W}_2$ followed by a $\tanh$ nonlinearity, so it is more expressive and can model nonlinear, complex interactions between $\mathbf{s}_t$ and $\mathbf{h}_i$; this often gives it an edge in quality, especially in settings where a single bilinear form is too restrictive.

Disadvantage: it is more computationally expensive — it requires two separate linear projections plus a $\tanh$ per (query, key) pair — whereas multiplicative attention reduces to efficient, hardware-optimized matrix multiplications, so additive attention is slower and has more parameters.

2. Analyzing NMT Systems (25 points)

- (a) (3 points) Look at the `src.vocab` file for some examples of phrases and words in the source language vocabulary. When encoding an input Mandarin Chinese sequence into “pieces” in the vocabulary, the tokenizer maps the sequence to a series of vocabulary items, each consisting of one or more characters (thanks to the `sentencepiece` tokenizer, we can perform this segmentation even when the original text has no white space). Given this information, how could adding a 1D Convolutional layer after the embedding layer and before passing the embeddings into the bidirectional encoder help our NMT system? **Hint:** each Mandarin Chinese character is either an entire word or a morpheme in a word. Look up the meanings of 电, 脑, and 电脑 separately for an example. The characters 电 (electricity) and 脑 (brain) when combined into the phrase 电脑 mean computer.

Solution: The 1D convolutional layer with kernel size 2 acts as a subword/character bigram composition layer. The CNN slides a length-2 window over the character (subword) embeddings, so it learns to combine pairs of adjacent characters into a higher-level representation that captures such compound/morphological meaning before the features reach the bidirectional LSTM. This gives the encoder input that already reflects word-level semantics rather than only raw character pieces, which helps the model translate multi-character Mandarin words more accurately.

- (b) (8 points) Here we present a series of errors we found in the outputs of our NMT model (which is the same as the one you just trained). For each example of a reference (i.e., ‘gold’) English translation, and NMT (i.e., ‘model’) English translation, please:

1. Identify the error in the NMT translation.
2. Provide possible reason(s) why the model may have made the error (either due to a specific linguistic construct or a specific model limitation).
3. Describe one possible way we might alter the NMT system to fix the observed error. There

are more than one possible fixes for an error. For example, it could be tweaking the size of the hidden layers or changing the attention mechanism.

Below are the translations that you should analyze as described above. Only analyze the underlined error in each sentence. Rest assured that you don't need to know Mandarin to answer these questions. You just need to know English! If, however, you would like some additional color on the source sentences, feel free to use a resource like https://www.archchinese.com/chinese_english_dictionary.html to look up words. Feel free to search the training data file to have a better sense of how often certain characters occur.

- i. (2 points) **Source Sentence:** 贼人其后被警方拘捕及被判处盗窃罪名成立。

Reference Translation: *the culprits were subsequently arrested and convicted.*

NMT Translation: *the culprit was subsequently arrested and sentenced to theft.*

Solution: A number/agreement error — the model produces the singular *the culprit was* where the reference has the plural *the culprits were*.

Possible reasons: Mandarin does not mark grammatical number, so 贼人 (“thief/criminal”) is ambiguous between singular and plural; the model must infer the number from context, and here it inferred (or the training distribution favored) the singular form. Recovering a feature that is unmarked in the source makes such agreement errors common.

Possible fix: Add a target-side signal that enforces agreement, e.g. a target-language language model used as a re-ranker over the beam which prefers the fluent plural *were* given a plural subject; or simply train on more data so the model learns to recover plurality from wider context.

- ii. (2 points) **Source Sentence:** 几乎已经没有地方容纳这些人, 资源已经用尽。

Reference Translation: *there is almost no space to accommodate these people, and resources have run out.*

NMT Translation: *the resources have been exhausted and resources have been exhausted.*

Solution: The model *omits* the first clause (“there is almost no space to accommodate these people”) and *repeats* the second clause, outputting “resources have been exhausted” twice; the translation is redundant and loses half the meaning.

Possible reasons: The source sentence has two clauses; the decoder's attention likely focused on the “resources” portion and lost track of the first clause, causing the first part to be dropped and the second to be regurgitated. This is a common failure for longer or multi-clause inputs, especially when attention does not explicitly remember which source parts have already been covered.

Possible fix: Use a *coverage* mechanism that tracks how much attention each source position has already received, penalizing re-attending to the same words and encouraging the model to cover the whole source sentence; alternatively, increase the beam size to avoid a single degenerate, repetitive hypothesis.

- iii. (2 points) **Source Sentence:** 当局已经宣布今天是国殇日。

Reference Translation: *authorities have announced a national mourning today.*

NMT Translation: *the administration has announced today's day.*

Solution: The compound word 国殇日 (“national mourning day”) is mistranslated as “today's day”; the model rendered 今天 (today) but failed to convey the core meaning “national mourning”, leaving a nonsensical, redundant “today's day”.

Possible reasons: 国殇 (“national mourning”) is a rare, formal compound that appears infrequently in the training data, so the model could not map it correctly and defaulted to translating the more common characters (今/天/日 as “today”/“day”). Rare and composite terms are a known weakness when they are underrepresented in the vocabulary and data.

Possible fix: Add more training data containing such rare formal compounds, or improve the subword segmentation so that 国殇 is treated as a recognizable unit; a larger model with more

capacity to store rare word senses could also help translate it correctly.

- iv. (2 points) **Source Sentence**⁴: 俗语有云:“唔做唔错”。

Reference Translation: “*act not, err not*”, so a saying goes.

NMT Translation: *as the saying goes, “it’s not wrong.”*

Solution: The Cantonese proverb 唔做唔错 (“if you don’t act, you won’t err” / “act not, err not”) is translated literally as “it’s not wrong”, losing the idiomatic meaning and the contrastive structure of the saying.

Possible reasons: The source is Cantonese, a distinct variety that is underrepresented relative to Mandarin in the training data, and 唔做唔错 is a fixed idiomatic expression whose meaning cannot be derived by translating its characters individually. Without exposure to such proverbs, the model falls back to a literal, semantically wrong rendering.

Possible fix: Include more Cantonese or idiomatic training data (or a phrase table of common idioms) so the model can learn the fixed expression; alternatively, a larger, more capable model that better captures multi-word fixed expressions would help render “act not, err not” idiomatically.

- (c) (14 points) BLEU score is the most commonly used automatic evaluation metric for NMT systems. It is usually calculated across the entire test set, but here we will consider BLEU defined for a single example.⁵ Suppose we have a source sentence $\mathbf{s}$, a set of k reference translations $\mathbf{r}_1, \dots, \mathbf{r}_k$, and a candidate translation $\mathbf{c}$. To compute the BLEU score of $\mathbf{c}$, we first compute the *modified n -gram precision* p_n of $\mathbf{c}$, for each of $n = 1, 2, 3, 4$, where n is the n in **n-gram**:

$$p_n = \frac{\sum_{\text{ngram} \in \mathbf{c}} \min \left(\max_{i=1, \dots, k} \text{Count}_{\mathbf{r}_i}(\text{ngram}), \text{Count}_{\mathbf{c}}(\text{ngram}) \right)}{\sum_{\text{ngram} \in \mathbf{c}} \text{Count}_{\mathbf{c}}(\text{ngram})} \quad (15)$$

Here, for each of the n -grams that appear in the candidate translation $\mathbf{c}$, we count the maximum number of times it appears in any one reference translation, capped by the number of times it appears in $\mathbf{c}$ (this is the numerator). We divide this by the number of n -grams in $\mathbf{c}$ (denominator).

Next, we compute the *brevity penalty* BP. Let $\text{len}(c)$ be the length of $\mathbf{c}$ and let $\text{len}(r)$ be the length of the reference translation that is closest to $\text{len}(c)$ (in the case of two equally-close reference translation lengths, choose $\text{len}(r)$ as the shorter one).

$$BP = \begin{cases} 1 & \text{if } \text{len}(c) \geq \text{len}(r) \\ \exp \left(1 - \frac{\text{len}(r)}{\text{len}(c)} \right) & \text{otherwise} \end{cases} \quad (16)$$

Lastly, the BLEU score for candidate $\mathbf{c}$ with respect to $\mathbf{r}_1, \dots, \mathbf{r}_k$ is:

$$BLEU = BP \times \exp \left(\sum_{n=1}^4 \lambda_n \log p_n \right) \quad (17)$$

where $\lambda_1, \lambda_2, \lambda_3, \lambda_4$ are weights that sum to 1. The log here is natural log.

⁴This is a Cantonese sentence! The data used in this assignment comes from GALE Phase 3, which is a compilation of news written in simplified Chinese from various sources scraped from the internet along with their translations. For more details, see <https://catalog.ldc.upenn.edu/LDC2017T02>.

⁵This definition of sentence-level BLEU score matches the `sentence_bleu()` function in the `nlTK` Python package. Note that the NLTK function is sensitive to capitalization. In this question, all text is lowercased, so capitalization is irrelevant. http://www.nltk.org/api/nltk.translate.html#nltk.translate.blu_score.sentence_bleu

- i. (5 points) Please consider this example:

Source Sentence s : 需要有充足和可预测的资源。

Reference Translation r_1 : *resources have to be sufficient and they have to be predictable*

Reference Translation r_2 : *adequate and predictable resources are required*

NMT Translation c_1 : there is a need for adequate and predictable resources

NMT Translation c_2 : resources be sufficient and predictable to

Please compute the BLEU scores for c_1 and c_2 . Let $\lambda_i = 0.5$ for $i \in \{1, 2\}$ and $\lambda_i = 0$ for $i \in \{3, 4\}$ (**this means we ignore 3-grams and 4-grams**, i.e., don't compute p_3 or p_4).

When computing BLEU scores, show your work (i.e., show your computed values for p_1 , p_2 , $\text{len}(c)$, $\text{len}(r)$ and BP). Note that the BLEU scores can be expressed between 0 and 1 or between 0 and 100. The code is using the 0 to 100 scale while in this question we are using the **0 to 1** scale. Please round your responses to 3 decimal places.

Which of the two NMT translations is considered the better translation according to the BLEU Score? Do you agree that it is the better translation?

Solution: With $\lambda_1 = \lambda_2 = 0.5$ and $\lambda_3 = \lambda_4 = 0$, we have $\text{BLEU} = BP \sqrt{p_1 p_2}$.

c_1 = “**there is a need for adequate and predictable resources**”. $\text{len}(c) = 9$; closest reference is r_1 with $\text{len}(r) = 11$. Unigrams: the only matches are adequate, and, predictable, resources $\Rightarrow p_1 = 4/9$. Bigrams: **adequate-and**, **and-predictable**, and **predictable-resources** appear in a reference $\Rightarrow p_2 = 3/8$. Since $9 < 11$, $BP = e^{1-11/9} = e^{-2/9} \approx 0.801$.

$$\text{BLEU}(c_1) = 0.801 \sqrt{(4/9)(3/8)} = 0.801 \sqrt{1/6} \approx 0.327.$$

c_2 = “**resources be sufficient and predictable to**”. $\text{len}(c) = 6$; closest reference is r_2 with $\text{len}(r) = 6$. Unigrams: all six match some reference (be, to appear in r_1) $\Rightarrow p_1 = 6/6 = 1$. Bigrams: **be-sufficient**, **sufficient-and**, **and-predictable** match $\Rightarrow p_2 = 3/5$. Since $\text{len}(c) = \text{len}(r)$, $BP = 1$.

$$\text{BLEU}(c_2) = \sqrt{3/5} \approx 0.775.$$

Per BLEU, c_2 (0.775) beats c_1 (0.327). I do *not* agree: c_1 is grammatical and complete, whereas c_2 is broken (missing “are required” and ending in a stray “to”). This illustrates that BLEU's surface n -gram overlap can reward matching words even in an ungrammatical, incomplete sentence.

- ii. (5 points) Our hard drive was corrupted and we lost Reference Translation r_1 . Please recompute BLEU scores for c_1 and c_2 , this time with respect to r_2 only. Which of the two NMT translations now receives the higher BLEU score? Do you agree that it is the better translation?

Solution: With only r_2 = “adequate and predictable resources are required” ($\text{len}(r) = 6$):

c_1 ($\text{len}(c) = 9$): unigram matches adequate, and, predictable, resources $\Rightarrow p_1 = 4/9$; bigram matches adequate-and, and-predictable, predictable-resources $\Rightarrow p_2 = 3/8$. Since $9 \geq 6$, $BP = 1$.

$$\text{BLEU}(c_1) = \sqrt{(4/9)(3/8)} = \sqrt{1/6} \approx 0.408.$$

c_2 ($\text{len}(c) = 6$): unigram matches resources, and, predictable $\Rightarrow p_1 = 3/6 = 0.5$; bigram match and-predictable only $\Rightarrow p_2 = 1/5$. Since $6 = 6$, $BP = 1$.

$$\text{BLEU}(c_2) = \sqrt{(0.5)(0.2)} = \sqrt{0.1} \approx 0.316.$$

Now c_1 (0.408) beats c_2 (0.316), and here I *agree*: c_1 is the better translation. The ranking flipped purely because the reference set changed, illustrating how strongly BLEU depends on the references used.

- iii. (2 points) Due to data availability, NMT systems are often evaluated with respect to only a single reference translation. Please explain (in a few sentences) why this may be problematic. In

your explanation, discuss how the BLEU score metric assesses the quality of NMT translations when there are multiple reference transitions versus a single reference translation.

Solution: A single reference is problematic because BLEU only credits the n -grams a candidate literally shares with the given reference, so a correct-but-differently-worded translation is under-scored merely for not matching that one reference. With multiple references, BLEU takes the *maximum* count of each n -gram across references, so a candidate matching any valid phrasing receives credit — a fairer estimate of quality.

- iv. (2 points) List two advantages and two disadvantages of BLEU, compared to human evaluation, as an evaluation metric for Machine Translation.

Solution:

Advantages: (1) automatic and fast — no human annotators are needed, so it is cheap, scalable to large test sets, and reproducible; (2) objective and consistent — it returns the same score every time, enabling easy, unambiguous comparison between systems without inter-annotator variability.

Disadvantages: (1) ignores semantics — it only measures surface n -gram overlap, so a fluent, correct translation phrased differently from the reference scores poorly, and semantic adequacy/fluency are not judged; (2) reference-dependent and gameable — it requires references, is sensitive to their number and quality, and can reward degenerate or over-short outputs (as the ungrammatical $\mathbf{c}_2$ scoring highest in part (i) shows).

- (d) (4 points) *Beam search* is often employed to improve the quality of machine translation systems. While you were training the model, beam search results for the same example sentence at different iterations were also recorded in TensorBoard, and accessible in the *TEXT* tab (Fig 3).

The recorded diagnostic information includes json documents with the following fields: **example_source** (the source sentence tokens), **example_target** (the ground truth target sentence tokens), and **hypotheses** (10 hypotheses corresponding to the search result with beam size 10). Note that a predicted translation is often called *hypothesis* in the neural machine translation jargon.

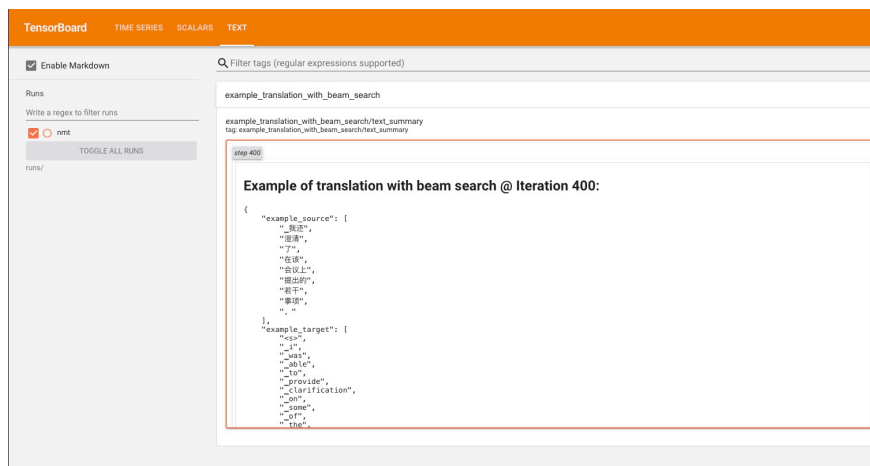


Figure 3: Translation with beam search results for an example sentence are recorded in tensorboard for various iterations. The same data is available in the `outputs/beam_search_diagnostics/` folder in your working directory.

- i. (2 points) Did the translation quality improve over the training iterations for the model? Give three examples of translations of the example sentence at iterations 200, 3000, and the last iteration to illustrate your answer. For each iteration, pick the first beam search hypothesis as an example:

Solution: In general, translation quality should improve over training: as the training loss decreases, the model learns better source–target alignments and more fluent output, so the first (highest-scoring) beam hypothesis at later iterations should be progressively more accurate and natural. I could not observe the actual hypotheses, since producing the `outputs/beam_search_diagnostics/` data requires training the model on a GPU, which I do not have; the above is therefore an inference rather than a report of observed translations.

- ii. (2 points) How do various hypotheses resulting from beam search qualitatively compare? Give three other examples of hypotheses proposed by beam search at the last iteration to illustrate your answer.

Solution: Beam search retains the top- k hypotheses by model score, so the 10 hypotheses should be ranked, near-synonymous variants of one another: they should share the same overall meaning and structure but can differ in word choice, function words, and word order. The first (highest-scoring) hypothesis should be the most likely and fluent, while lower-ranked ones should be slightly less likely but still plausible. Again this is an inference; the observed hypotheses require a trained model, which I cannot run without a GPU.

Submission Instructions

You shall submit this assignment on GradeScope as two submissions – one for “Assignment 3 [coding]” and another for “Assignment 3 [written]”:

1. Run the `collect_submission.sh` script on the cloud VM to produce your `assignment3.zip` file. See *How to Download the Gradescope Submission Package from Your Cloud VM* in the GCP How-to Appendix.
2. Upload your `assignment3.zip` file to GradeScope to “Assignment 3 [coding]”.
3. Upload your written solutions to GradeScope to “Assignment 3 [written]”. When you submit your assignment, make sure to tag all the pages for each problem according to Gradescope’s submission directions. Points will be deducted if the submission is not correctly tagged.

Appendix: GCP How-to

How to Obtain SSH connection info After following the instructions in the [GCP Guide for CS224n](#), your Google Cloud Compute Engine dashboard should look like Fig 4. Click on the arrow button near SSH and select *View gcloud command* as shown in the figure. What you will get looks like the following. Make a note of the zone, the machine name, and the project name which in the example are respectively `us-west4-b`, `nvidia-gpu-optimized-vmi-1-vm` and `grand-hangar-420500`. **Yours will be different!**

```
gcloud compute ssh --zone "us-west4-b" "nvidia-gpu-optimized-vmi-1-vm" --
  ↪ project "grand-hangar-420500"
```

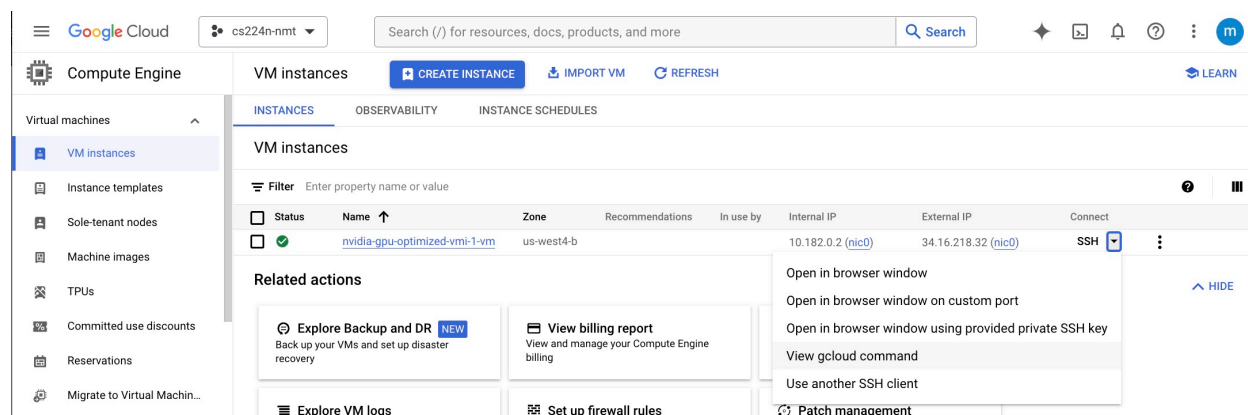


Figure 4: The cloud VM you create should be based on the `nvidia-gpu-optimized-vmi` virtual machine image (VMI). After deploying the instance, click on the menu near SSH and select *View gcloud command* to obtain the SSH connection information.

How to Connect to the Cloud VM with SSH Tunelling The following ssh connection command, which includes the `--ssh-flag "-L 6007:localhost:6007"` option, will also forward port 6007 of the cloud machine to the same port number on your local machine (ssh tunneling). Therefore, if the tensorboard server is running on the cloud VM and listening to port 6007, after establishing an ssh connection as follows, you will be able to access at <http://localhost:6007/> in a web browser on your local machine. Run the following ssh command to connect to the cloud machine. **be sure to use your own zone, project name and machine name**

```
gcloud compute ssh --zone "us-west4-b" "nvidia-gpu-optimized-vmi-1-vm" --
  ↪ project "grand-hangar-420500" --ssh-flag "-L 6007:localhost:6007"
```

How to Copy Your Code From Your Computer to the Cloud VM

- zip current directory, which should contain the `env-cpu.yml` and `env-gpu.yml` files.

```
zip -r student.zip *
```

- copy the obtained zip file to the cloud machine

```
gcloud compute scp student.zip nvidia-gpu-optimized-vmi-1-vm:~/ --zone "
  ↪ us-west4-b" --project "grand-hangar-420500"
```

How to Train the NMT System on the Cloud VM

- If zip is not installed on your cloud machine. Install it as follows and try again.

```
sudo apt-get install zip
```

- Unzip student folder on the cloud machine

```
unzip student.zip -d student
cd student
```

- Create conda GPU environment

```
conda env create --file env-gpu.yml
```

- Activate conda GPU environment

```
conda activate cs224n-nmt-gpu
```

- create a tmux session in which you start training the machine translation model

```
tmux new -s s-nmt
```

```
# start the training (in the tmux session)
sh run.sh train
```

```
# detach from the tmux session
CTRL+B D
```

- create a tmux session in which you start TensorBoard

```
tmux new -s s-tboard
```

```
# start the tensorboard (in the tmux session)
tensorboard --logdir runs/ --port 6007
```

```
# detach from the tmux session
CTRL+B D
```

- You can attach/detach from the two tmux sessions as needed. See the appendix on Tmux for basic use cases.

- If you established the SSH connection with the appropriate port forwarding options, you can view tensorboard in a web browser on your local machine at this address <http://localhost:6007/> (Fig. 5)

- when the training is complete, run the test phase

```
tmux attach -t s-nmt
sh run.sh test
```

- The next section shows how to make a grade scope submission with the artifacts generated on the cloud VM.

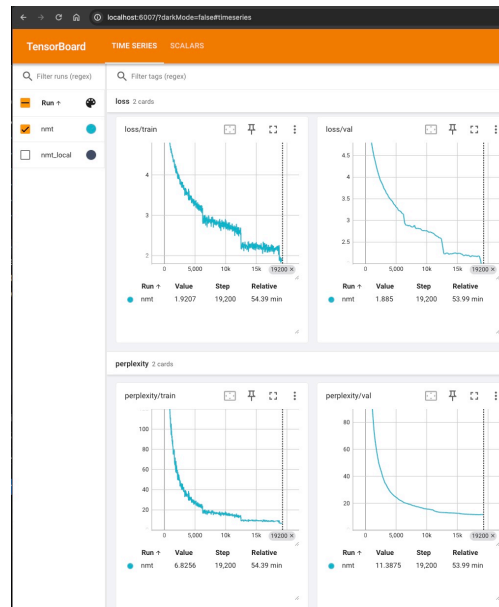


Figure 5: Tensorboard running on your local machine and showing loss and perplexity values on your cloud virtual machine.

How to Download the Gradescope Submission Package from Your Cloud VM

- Connect to the cloud VM as specified above
- On the Cloud VM, run the following command to create the gradescope submission package (**assignment3.zip**)

```
sh collect_submission.sh
```
- Disconnect from the cloud VM (by running **exit**), or open a terminal on your local machine.
- Download the GradeScope submission package to your local machine

```
gcloud compute scp nvidia-gpu-optimized-vmi-1-vm:~/student/assignment3.zip  

  ↪ . --zone "us-west4-b" --project "grand-hangar-420500"
```
- Submit **assignment3.zip** to GradeScope

Appendix: Tmux How-to

tmux (terminal multiplexer) is a tool that lets you create and toggle between multiple terminals that run in the background, and keep them running even after you disconnect your ssh session. The following assumes that you are already connected (via ssh) to your cloud virtual machine instance, and shows you basic use cases of tmux. Learn more about tmux at <https://github.com/tmux/tmux/wiki>

- Create and attach to a new tmux session (named *s-tboard*)

```
tmux new -s s-tboard
```

- Run a program in the new session. We will run tensorboard

```
tensorboard --logdir runs/ --port 6007
```

- (From within the tmux session), Detach from a tmux session, and leave the program you started running in the session.

```
CTRL+b d
```

- (After detaching from tmux), view all tmux sessions

```
tmux ls
```

- attach to a particular tmux session (named s-nmt)

```
tmux attach -t s-nmt
```

- to terminate a tmux session (from within the session)

```
exit
```